programmingworld.tech

Beyond Django CRUD

Implementing Advanced Query Optimization in Django ORM

Django's ORM makes database interactions seamless, allowing developers to write queries in Python without **raw SQL**. However, as **applications scale**, **inefficient queries** can **slow down performance**, leading to high latency and database load.

This guide explores **advanced query optimization techniques** in Django ORM to go beyond basic CRUD (Create, Read, Update, Delete) operations and improve efficiency.

1. Use QuerySet Caching to Avoid Repeated Queries

Using cache reduces redundant queries for frequently accessed data.

```python
from django.core.cache import cache

users = cache.get("all_users")
if not users:
    users = list(User.objects.all())
    # Cache for 10 minutes
    cache.set("all_users", users, timeout=600)
```

Caching helps reduce repeated database hits.

2. Avoid .count() on Large Datasets

Using .count() on large tables can be expensive.

❌ Inefficient Way:

```
# Executes a full table scan
if User.objects.count() > 0:
    print("Users exist")
```

✅ Optimized Way (.exists() is Faster):

```
# Checks only one record
if User.objects.exists():
    print("Users exist")
```

3. Use Indexes for Faster Lookups

Indexes speed up queries on frequently filtered fields.

✅ Add **db_index=True** for frequently queried fields:

```
class Product(models.Model):
    name = models.CharField(
                    max_length=255, db_index=True)
```

Use indexes on fields frequently used in filter() and order_by() queries.

4. Optimize Bulk Inserts and Updates

Performing operations on multiple records one by one is inefficient.

Use **bulk_create()** for mass inserts:

```
users = [User(name=f"User {i}") for i in range(1000)]
# Executes in a single query
User.objects.bulk_create(users)
```

Bulk operations significantly reduce query execution time.

5. Reduce Query Count with only() and defer()

By default, Django loads all model fields, which can be slow for large tables.

✅ Use **only()** to fetch specific fields:

```
# Loads only 'id' and 'name'
users = User.objects.only("id", "name")
```

✅ Use **defer()** to exclude heavy fields:

```
# Loads all fields except 'profile_picture'
users = User.objects.defer("profile_picture")
```

programmingworld.tech

Want to Know More Queries

Find a detailed blog on my Medium channel

LINK IN BIO